8.1.2 간단한 팔렛 개발하기

개요

이 튜토리얼에서는 Substrate의 핵심 구성 요소인 팔렛(Pallet)을 직접 개발해보겠습니다. 간단한 “Hello World” 팔렛부터 시작하여 상태 저장, 이벤트 발생, 오류 처리까지 팔렛 개발의 기본 개념들을 실습을 통해 학습합니다.

학습 목표

- 팔렛의 기본 구조와 구성 요소 이해
- 스토리지 아이템 정의 및 사용법
- 디스패처블 함수(Extrinsic) 구현
- 이벤트와 오류 처리 방법
- 팔렛을 런타임에 통합하는 과정

사전 요구사항

- 첫 번째 Substrate 노드 실행 완료
- Rust 기본 문법 이해
- Substrate 노드 템플릿 프로젝트

1단계: 팔렛 기본 구조 이해

1.1 팔렛 구성 요소

Substrate 팔렛은 다음 주요 구성 요소들로 이루어집니다:

```
// 팔렛의 기본 구조
#[frame_support::pallet]
pub mod pallet {
use frame_support::pallet_prelude::*;
use frame_system::pallet_prelude::*;

// 1. 설정 트레이트
#[pallet::config]
pub trait Config: frame_system::Config {
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
}

// 2. 팔렛 구조체
#[pallet::pallet]
pub struct Pallet<T>(_);

// 3. 스토리지 아이템
#[pallet::storage]
pub type Something<T> = StorageValue<_, u32>;

// 4. 이벤트
```

```rust
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub enum Event<T: Config> {
SomethingStored { something: u32, who: T::AccountId },
}

// 5. 오류
#[pallet::error]
pub enum Error<T> {
NoneValue,
StorageOverflow,
}

// 6. 디스패처블 함수들
#[pallet::call]
impl<T: Config> Pallet<T> {
#[pallet::weight(10_000)]
pub fn do_something(origin: OriginFor<T>, something: u32) -> DispatchResult {
// 구현 내용
Ok(())
}
}
```

```
}
```

1.2 기존 템플릿 팔렛 확인

```
# 템플릿 팔렛 구조 확인
cd ~/substrate-projects/substrate-node-template
ls -la pallets/template/src/
cat pallets/template/src/lib.rs
```

2단계: 첫 번째 커스텀 팔렛 생성

2.1 새 팔렛 디렉토리 생성

```
# 새 팔렛 디렉토리 생성
mkdir -p pallets/hello-substrate/src
cd pallets/hello-substrate
```

2.2 Cargo.toml 파일 생성

```
# pallets/hello-substrate/Cargo.toml
[package]
name = "pallet-hello-substrate"
version = "4.0.0-dev"
```

```
description = "Hello Substrate Pallet"

authors = ["Substrate DevHub <https://github.com/substrate-developer-hub>"]

homepage = "https://substrate.io"

edition = "2021"

license = "MIT-0"

publish = false

repository = "https://github.com/substrate-developer-hub/substrate-node-template



[package.metadata.docs.rs]

targets = ["x86_64-unknown-linux-gnu"]



[dependencies]

codec = { package = "parity-scale-codec", version = "3.6.1", default-features =

"derive",

] }

scale-info = { version = "2.5.0", default-features = false, features = ["derive"



frame-benchmarking = { version = "4.0.0-dev", default-features = false, optional

frame-support = { version = "4.0.0-dev", default-features = false, git = "https:

frame-system = { version = "4.0.0-dev", default-features = false, git = "https:/



[dev-dependencies]

sp-core = { version = "21.0.0", git = "https://github.com/paritytech/polkadot-sd
```

```
sp-io = { version = "23.0.0", git = "https://github.com/paritytech/polkadot-sdk.

sp-runtime = { version = "24.0.0", git = "https://github.com/paritytech/polkadot


[features]

default = ["std"]

std = [

"codec/std",

"frame-benchmarking?/std",

"frame-support/std",

"frame-system/std",

"scale-info/std",

]

runtime-benchmarks = ["frame-benchmarking/runtime-benchmarks"]

try-runtime = ["frame-support/try-runtime"]
```

2.3 기본 팔렛 구현

```
// pallets/hello-substrate/src/lib.rs

#![cfg_attr(not(feature = "std"), no_std)]


pub  use pallet::*;


#[frame_support::pallet]
```

```rust
pub mod pallet {
use frame_support::pallet_prelude::*;
use frame_system::pallet_prelude::*;

#[pallet::pallet]
pub struct Pallet<T>(_);

/// 팔렛 설정 트레이트
#[pallet::config]
pub trait Config: frame_system::Config {
/// 런타임에서 발생하는 이벤트 타입
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::
}

/// 간단한 값을 저장하는 스토리지
#[pallet::storage]
#[pallet::getter(fn something)]
pub type Something<T> = StorageValue<_, u32>;

/// 각 계정별로 값을 저장하는 스토리지 맵
#[pallet::storage]
#[pallet::getter(fn user_values)]
pub type UserValues<T: Config> = StorageMap<_, Blake2_128Concat, T::AccountId,
```

```
/// 팔렛에서 발생하는 이벤트들
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub  enum Event<T: Config> {
/// 값이 저장되었을 때 발생하는 이벤트 [value, who]
SomethingStored { something: u32, who: T::AccountId },
/// 사용자별 값이 설정되었을 때 발생하는 이벤트 [user, value]
UserValueSet { user: T::AccountId, value: u32 },
/// 값이 제거되었을 때 발생하는 이벤트 [who]
ValueRemoved { who: T::AccountId },
}


/// 팔렛에서 발생할 수 있는 오류들
#[pallet::error]
pub  enum Error<T> {
/// 저장된 값이 없음
NoneValue,
/// 저장소 오버플로우
StorageOverflow,
/// 값이 너무 큼
ValueTooLarge,
}
```

```rust
/// 팔렛의 디스패처블 함수들
#[pallet::call]
impl<T: Config> Pallet<T> {
/// 간단한 값을 저장하는 함수
#[pallet::call_index(0)]
#[pallet::weight(10_000)]
pub fn do_something(origin: OriginFor<T>, something: u32) -> DispatchResult {
// 트랜잭션이 서명된 계정에서 왔는지 확인
let who = ensure_signed(origin)?;

// 값이 1000을 초과하지 않는지 확인
ensure!(something <= 1000, Error::<T>::ValueTooLarge);

// 스토리지에 값 저장
<Something<T>>::put(&something);

// 이벤트 발생
Self::deposit_event(Event::SomethingStored { something, who });

Ok(())
}
```

```
/// 사용자별 값을 설정하는 함수
#[pallet::call_index(1)]
#[pallet::weight(10_000)]
pub fn set_user_value(origin: OriginFor<T>, value: u32) -> DispatchResult {
let who = ensure_signed(origin)?;

// 값 유효성 검사
ensure!(value <= 10000, Error::<T>::ValueTooLarge);

// 사용자별 스토리지에 값 저장
<UserValues<T>>::insert(&who, &value);

// 이벤트 발생
Self::deposit_event(Event::UserValueSet { user: who, value });

Ok(())
}

/// 사용자의 값을 제거하는 함수
#[pallet::call_index(2)]
#[pallet::weight(10_000)]
pub fn remove_user_value(origin: OriginFor<T>) -> DispatchResult {
let who = ensure_signed(origin)?;
```

```
// 값이 존재하는지 확인
ensure!(<UserValues<T>>::contains_key(&who), Error::<T>::NoneValue);

// 값 제거
<UserValues<T>>::remove(&who);

// 이벤트 발생
Self::deposit_event(Event::ValueRemoved { who });

Ok(())
}

/// 저장된 값을 1 증가시키는 함수
#[pallet::call_index(3)]
#[pallet::weight(10_000)]
pub fn increment_value(origin: OriginFor<T>) -> DispatchResult {
let _who = ensure_signed(origin)?;

// 현재 값 가져오기
let current = Self::something().unwrap_or(0);

// 오버플로우 확인
```

```
let new_value = current.checked_add(1).ok_or(Error::<T>::StorageOverflow)?;

// 새 값 저장
<Something<T>>::put(&new_value);

Ok(())
}
}

/// 팔렛의 헬퍼 함수들
impl<T: Config> Pallet<T> {
/// 현재 저장된 값을 반환하는 함수
pub fn get_current_value() -> Option<u32> {
Self::something()
}

/// 특정 사용자의 값을 반환하는 함수
pub fn get_user_value(user: &T::AccountId) -> Option<u32> {
Self::user_values(user)
}
}
}
```

3단계: 팔렛을 런타임에 통합

3.1 런타임 Cargo.toml 수정

```
# runtime/Cargo.toml의 [dependencies] 섹션에 추가

pallet-hello-substrate = { version = "4.0.0-dev", default-features = false, path


# [features] 섹션의 std 배열에 추가

std = [

# ... 기존 항목들

"pallet-hello-substrate/std",

]
```

3.2 런타임 구성 수정

```
// runtime/src/lib.rs에 추가


// 팔렛 설정 구현

impl pallet_hello_substrate::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

}


// construct_runtime! 매크로에 팔렛 추가
```

```
construct_runtime!(
pub struct Runtime {
// ... 기존 팔렛들
HelloSubstrate: pallet_hello_substrate,
}
);
```

3.3 런타임 빌드 및 테스트

```
# 런타임 빌드
cd ~/substrate-projects/substrate-node-template
cargo build --release

# 빌드 성공 확인
echo "빌드 완료: $?"
```

4단계: 팔렛 기능 테스트

4.1 노드 재시작

```
# 기존 노드 중지 (Ctrl+C)
# 새 노드 시작
./target/release/node-template --dev --tmp
```

4.2 Polkadot.js Apps에서 팔렛 확인

1. https://polkadot.js.org/apps/ 접속
2. “Developer” → “Extrinsics” 메뉴로 이동
3. “submit the following extrinsic” 드롭다운에서 “helloSubstrate” 확인
4. 사용 가능한 함수들 확인:

- `doSomething(something)`
- `setUserValue(value)`
- `removeUserValue()`
- `incrementValue()`

4.3 기본 함수 테스트

doSomething 함수 테스트:

1. Extrinsics 페이지에서:

- pallet: `helloSubstrate`
- callable: `doSomething`
- something: `42`

2. “Submit Transaction” 클릭
3. “Explorer” 탭에서 이벤트 확인

예상 이벤트:

```
helloSubstrate.SomethingStored

something: 42

who: 5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY
```

4.4 스토리지 상태 확인

1. “Developer” → “Chain state” 메뉴로 이동
2. “selected state query” 드롭다운에서:

- pallet: `helloSubstrate`
- storage: `something(): Option<u32>`

3. “+” 버튼 클릭하여 쿼리 실행
4. 저장된 값 `42` 확인

4.5 사용자별 값 테스트

setUserValue 함수 테스트:

1. Extrinsics에서:

- pallet: `helloSubstrate`
- callable: `setUserValue`
- value: `100`

2. 트랜잭션 제출

사용자 값 확인:

1. Chain state에서:

- pallet: `helloSubstrate`
- storage: `userValues(AccountId32): Option<u32>`
- AccountId32: Alice 계정 주소 입력

2. 저장된 값 `100` 확인

5단계: 고급 기능 추가

5.1 제네시스 설정 추가

```
// pallets/hello-substrate/src/lib.rs에 추가

/// 제네시스 설정
#[pallet::genesis_config]
#[derive(frame_support::DefaultNoBound)]
pub struct GenesisConfig<T: Config> {
pub initial_value: Option<u32>,
pub user_values: Vec<(T::AccountId, u32)>,
}

#[pallet::genesis_build]
impl<T: Config> BuildGenesisConfig for GenesisConfig<T> {
fn build(&self) {
// 초기값 설정
if let Some(value) = self.initial_value {
<Something<T>>::put(value);
}

// 사용자별 초기값 설정
for (user, value) in &self.user_values {
<UserValues<T>>::insert(user, value);
```

```
}
}
}
```

5.2 런타임에 제네시스 설정 추가

```
// runtime/src/lib.rs의 제네시스 설정에 추가
hello_substrate: HelloSubstrateConfig {
initial_value: Some(100),
user_values: vec![],
},
```

5.3 벤치마킹 추가

```
// pallets/hello-substrate/src/benchmarking.rs 생성
#![cfg(feature = "runtime-benchmarks")]

use  super::*;
use frame_benchmarking::v2::*;
use frame_system::RawOrigin;

#[benchmarks]
mod benchmarks {
```

```
use super::*;

#[benchmark]
fn do_something() {
let caller: T::AccountId = whitelisted_caller();
let value = 42u32;

#[extrinsic_call]
do_something(RawOrigin::Signed(caller), value);

assert_eq!(Something::<T>::get(), Some(value));
}

#[benchmark]
fn set_user_value() {
let caller: T::AccountId = whitelisted_caller();
let value = 100u32;

#[extrinsic_call]
set_user_value(RawOrigin::Signed(caller.clone()), value);

assert_eq!(UserValues::<T>::get(&caller), Some(value));
}
```

```
impl_benchmark_test_suite!(Pallet, crate::mock::new_test_ext(), crate::mock::Tes

}
```

6단계: 단위 테스트 작성

6.1 테스트 모듈 생성

```
// pallets/hello-substrate/src/tests.rs

use  super::*;

use  crate::{mock::*, Error, Event};

use frame_support::{assert_noop, assert_ok, traits::Get};


#[test]

fn  it_works_for_default_value() {

new_test_ext().execute_with(|| {

// 초기 상태에서는 값이 없어야 함

assert_eq!(HelloSubstrate::something(), None);

});

}


#[test]

fn  do_something_works() {
```

```
new_test_ext().execute_with(|| {
// 값 저장 테스트
assert_ok!(HelloSubstrate::do_something(RuntimeOrigin::signed(1), 42));
assert_eq!(HelloSubstrate::something(), Some(42));


// 이벤트 확인
System::assert_last_event(Event::SomethingStored { something: 42, who: 1 }.into(
});
}


#[test]
fn  do_something_fails_for_large_value() {
new_test_ext().execute_with(|| {
// 큰 값에 대해 실패해야 함
assert_noop!(
HelloSubstrate::do_something(RuntimeOrigin::signed(1), 1001),
Error::<Test>::ValueTooLarge
);
});
}


#[test]
fn  set_user_value_works() {
```

```rust
new_test_ext().execute_with(|| {
// 사용자 값 설정 테스트
assert_ok!(HelloSubstrate::set_user_value(RuntimeOrigin::signed(1), 100));
assert_eq!(HelloSubstrate::user_values(1), Some(100));

// 이벤트 확인
System::assert_last_event(Event::UserValueSet { user: 1, value: 100 }.into());
});
}

#[test]
fn  remove_user_value_works() {
new_test_ext().execute_with(|| {
// 먼저 값 설정
assert_ok!(HelloSubstrate::set_user_value(RuntimeOrigin::signed(1), 100));
assert_eq!(HelloSubstrate::user_values(1), Some(100));

// 값 제거
assert_ok!(HelloSubstrate::remove_user_value(RuntimeOrigin::signed(1)));
assert_eq!(HelloSubstrate::user_values(1), None);

// 이벤트 확인
System::assert_last_event(Event::ValueRemoved { who: 1 }.into());
```

```
});
}

#[test]
fn  remove_user_value_fails_when_no_value() {
new_test_ext().execute_with(|| {
// 값이 없을 때 제거 시도하면 실패해야 함
assert_noop!(
HelloSubstrate::remove_user_value(RuntimeOrigin::signed(1)),
Error::<Test>::NoneValue
);
});
}

#[test]
fn  increment_value_works() {
new_test_ext().execute_with(|| {
// 초기값 설정
assert_ok!(HelloSubstrate::do_something(RuntimeOrigin::signed(1), 5));
assert_eq!(HelloSubstrate::something(), Some(5));

// 값 증가
assert_ok!(HelloSubstrate::increment_value(RuntimeOrigin::signed(1)));
```

```
assert_eq!(HelloSubstrate::something(), Some(6));
});
}
```

6.2 Mock 런타임 생성

```
// pallets/hello-substrate/src/mock.rs
use crate as pallet_hello_substrate;
use frame_support::{
derive_impl,
traits::{ConstU16, ConstU64},
};
use sp_core::H256;
use sp_runtime::{
traits::{BlakeTwo256, IdentityLookup},
BuildStorage,
};

type Block = frame_system::mocking::MockBlock<Test>;

frame_support::construct_runtime!(
pub enum Test {
System: frame_system,
```

```
HelloSubstrate: pallet_hello_substrate,

}

);



#[derive_impl(frame_system::config_preludes::TestDefaultConfig as frame_system::

impl frame_system::Config for Test {

type BaseCallFilter = frame_support::traits::Everything;

type BlockWeights = ();

type BlockLength = ();

type DbWeight = ();

type RuntimeOrigin = RuntimeOrigin;

type RuntimeCall = RuntimeCall;

type Nonce = u64;

type Hash = H256;

type Hashing = BlakeTwo256;

type AccountId = u64;

type Lookup = IdentityLookup<Self::AccountId>;

type Block = Block;

type RuntimeEvent = RuntimeEvent;

type BlockHashCount = ConstU64<250>;

type Version = ();

type PalletInfo = PalletInfo;

type AccountData = ();
```

```
type OnNewAccount = ();

type OnKilledAccount = ();

type SystemWeightInfo = ();

type SS58Prefix = ConstU16<42>;

type OnSetCode = ();

type MaxConsumers = frame_support::traits::ConstU32<16>;

}



impl pallet_hello_substrate::Config for Test {

type RuntimeEvent = RuntimeEvent;

}



pub  fn  new_test_ext() -> sp_io::TestExternalities {

frame_system::GenesisConfig::<Test>::default().build_storage().unwrap().into()

}
```

6.3 lib.rs 파일 업데이트

```
// pallets/hello-substrate/src/lib.rs 상단에 추가

#![cfg_attr(not(feature = "std"), no_std)]



pub  use pallet::*;
```

```
#[cfg(test)]
mod mock;

#[cfg(test)]
mod tests;

#[cfg(feature = "runtime-benchmarks")]
mod benchmarking;

// 기존 팔렛 코드...
```

6.4 테스트 실행

```
# 팔렛 테스트 실행
cd ~/substrate-projects/substrate-node-template
cargo test -p pallet-hello-substrate

# 전체 프로젝트 테스트
cargo test
```

예상 출력:

```
running 6 tests
test tests::it_works_for_default_value ... ok
```

```
test tests::do_something_works ... ok

test tests::do_something_fails_for_large_value ... ok

test tests::set_user_value_works ... ok

test tests::remove_user_value_works ... ok

test tests::remove_user_value_fails_when_no_value ... ok

test tests::increment_value_works ... ok



test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

7단계: 실제 사용 시나리오 테스트

7.1 복합 시나리오 테스트

Polkadot.js Apps에서 다음 시나리오를 테스트해보세요:

시나리오 1: 기본 값 저장 및 조회

1. `doSomething(42)` 실행
2. Chain state에서 `something()` 값 확인
3. `incrementValue()` 실행
4. 값이 43으로 증가했는지 확인

시나리오 2: 사용자별 값 관리

1. Alice로 `setUserValue(100)` 실행
2. Bob으로 `setUserValue(200)` 실행
3. 각각의 값이 올바르게 저장되었는지 확인

4. Alice로 `removeUserValue()` 실행

5. Alice의 값만 제거되었는지 확인

시나리오 3: 오류 처리 테스트

1. `doSomething(1001)` 실행 → 실패해야 함

2. 값을 설정하지 않은 계정으로 `removeUserValue()` 실행 → 실패해야 함

7.2 이벤트 모니터링

```
# 별도 터미널에서 이벤트 모니터링
curl -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method": "chain_subscribeNewHeads", "params":[]}'
ws://localhost:9944
```

8단계: 문제 해결 및 디버깅

8.1 일반적인 컴파일 오류

오류 1: 트레이트 바운드 누락

```
// 오류 메시지: trait bound `T: Config` is not satisfied
// 해결: 제네릭 타입에 올바른 바운드 추가
impl<T: Config> Pallet<T> {
// ...
}
```

오류 2: 이벤트 타입 불일치

```
// 오류 메시지: expected `Event<T>`, found `Event<Test>`
// 해결: 런타임 설정에서 올바른 이벤트 타입 지정
type RuntimeEvent = RuntimeEvent;
```

8.2 런타임 통합 문제

문제: 팔렛이 런타임에서 인식되지 않음

1. `runtime/Cargo.toml` 에 의존성 추가 확인
2. `construct_runtime!` 매크로에 팔렛 추가 확인
3. 팔렛 Config 구현 확인

8.3 디버깅 도구 활용

```
// 디버그 출력 추가
use frame_support::log;

#[pallet::call]
impl<T: Config> Pallet<T> {
pub fn do_something(origin: OriginFor<T>, something: u32) -> DispatchResult {
let who = ensure_signed(origin)?;
// 디버그 로그
log::info!("do_something called by {:?} with value {}", who, something);
// ... 나머지 구현
```

```
}

}
```

9단계: 다음 단계 및 확장

9.1 추가 기능 아이디어

- **권한 관리**: 특정 계정만 값을 설정할 수 있도록 제한
- **값 검증**: 더 복잡한 비즈니스 로직 추가
- **배치 처리**: 여러 값을 한 번에 처리하는 함수
- **시간 기반 로직**: 블록 번호나 타임스탬프 기반 기능

9.2 성능 최적화

```
// 가중치 최적화
#[pallet::weight(T::DbWeight::get().reads_writes(1, 1))]
pub fn optimized_function(origin: OriginFor<T>) -> DispatchResult {
// 실제 데이터베이스 읽기/쓰기 횟수에 맞춘 가중치
}
```

9.3 다음 학습 주제

- 팔렛 간 상호작용 (Tight/Loose Coupling)
- 오프체인 워커 (Off-chain Workers)
- 런타임 API 구현

- 마이그레이션 및 업그레이드

요약

이 튜토리얼에서는 다음을 학습했습니다:

1. **팔렛 구조**: Config, Storage, Events, Errors, Calls의 기본 구성
2. **스토리지 사용**: StorageValue와 StorageMap을 통한 데이터 저장
3. **함수 구현**: 디스패처블 함수와 오류 처리
4. **런타임 통합**: 팔렛을 런타임에 추가하는 과정
5. **테스트 작성**: 단위 테스트와 Mock 런타임 구성
6. **실제 사용**: Polkadot.js Apps를 통한 기능 테스트

다음 튜토리얼에서는 이 기본 팔렛을 바탕으로 파라체인 템플릿을 사용하는 방법을 학습하겠습니다.

참고 자료

- FRAME 팔렛 개발 가이드
- Substrate 매크로 참조
- 팔렛 예제 모음
- Polkadot.js API 문서